

```

#include <iostream>
#include <vector>
#include <omp.h>
using namespace std;

// Sequential Bubble Sort
void sequentialBubbleSort(vector<int> &arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}

// Parallel Bubble Sort (Odd-Even Transposition)
void parallelBubbleSort(vector<int> &arr) {
    int n = arr.size();

    for (int i = 0; i < n; i++) {
        // Even phase
        #pragma omp parallel for
        for (int j = 0; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }

        // Odd phase
        #pragma omp parallel for
        for (int j = 1; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    vector<int> arr(n), arr_copy;

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    arr_copy = arr;
}

```

```

// Sequential timing
double start = omp_get_wtime();
sequentialBubbleSort(arr);
double end = omp_get_wtime();

cout << "\nSequential Sorted Array: ";
for (int x : arr) cout << x << " ";
cout << "\nTime (Sequential): " << (end - start) << "
seconds\n";

// Parallel timing
start = omp_get_wtime();
parallelBubbleSort(arr_copy);
end = omp_get_wtime();

cout << "\nParallel Sorted Array: ";
for (int x : arr_copy) cout << x << " ";
cout << "\nTime (Parallel): " << (end - start) << " seconds\n";

return 0;
}

```